Szoftertechnológiák zh I

Mi a szoftver?
- Számítógépes programok és a hozzá kapcsolódó dokumentációk (pl. követelmények, tervezési modellek és felhasználói kézikönyvek)

Mi a szoftverek két alapvető típusa?
- Általános (generic): felhasználók széles rétege számára fejlesztett és általuk használt szoftver. Pl. Excel, Word
- Egyedi (custom): egy megrendelő egyedi igényei szerint készült

Sorolja fel a szoftvergyártás 4 alapvető lépését.
- Specifikáció: mit kell a rendszernek tudnia és mik a fejlesztési kényszerek, kötöttségek
- Fejlesztés (tervezés és implementáció): A szoftver rendszer megalkotása
- Validáció (ellenőrzés): a szoftver azt csinálja, amit a megrendelő akar?
- Evolúció: A szoftver változó igények szerinti továbbfejlesztése.

Mit jelent a CASE betűszó?
- Computer-Aided Software Engineering. Olyan szoftver rendszerek, amelyek a szoftverfejlesztési folyamatot automatikus eszközökkel támogatják. Upper-CASE (korai fázisát támogató eszközök), Lower-CASE (későbbi fázisát támogató eszközök)

Mit jelent a „legacy rendszer”?
- Olyan ember-gép rendszerek, amelyek régi vagy elavult technológiával lettek kifejlesztve.

Sorolja fel a jó szoftver 4 alapvető ismérvét.
- Karbantarthatóság: a szoftvernek a változó igényekkel lépést kell tartania
- Megbízhatóság: a szoftvernek megbízhatóan kell működnie
- Hatékonyság: ne bánjon pazarlóan a rendszererőforrásokkal
- Befogadhatóság/használhatóság: a célfelhasználók által befogadhatónak kell lennie: érthető, könnyen használható, más rendszerekkel kompatibilis.

Miért nehéz a szoftverfejlesztés? Soroljon fel 3 indokot.
- Heterogenitás: szoftverkészítést heterogén platformokra és végrehajtási környezetekre.
- Határidők: gyorsabb fejlesztés és átadás.
- Bizalom: felhasználók bizalmát megnyerni képes fejlesztési technológia.

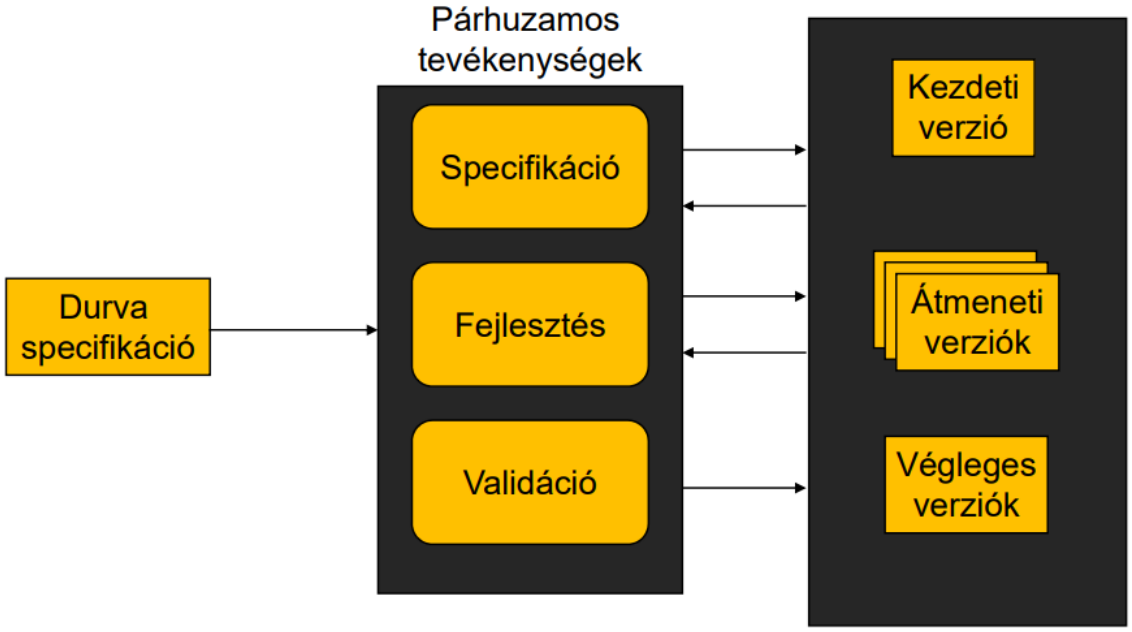
Milyen típusú információkat tartalmaznak a szoftverfejlesztési módszertanok?
- Modell leírások: a létrehozandó grafikus modellek leírása
- Szabályok: a rendszermodellekre vonatkozó kényszerek
- Ajánlások: a helyes tervezési megoldásokra vonatkozó tanácsok
- Fejlesztési útmutató: a modellfejlesztés során végrehajtandó tevékenységek sorozata

Sorolja fel a vízesés modell 5 lépését.
- Követelményanalízis és -definíció, rendszer- és szoftvertervezés, implementáció és a részegységek tesztelése, részegységek integrálása és a rendszer tesztelés, működtetés és karbantartás

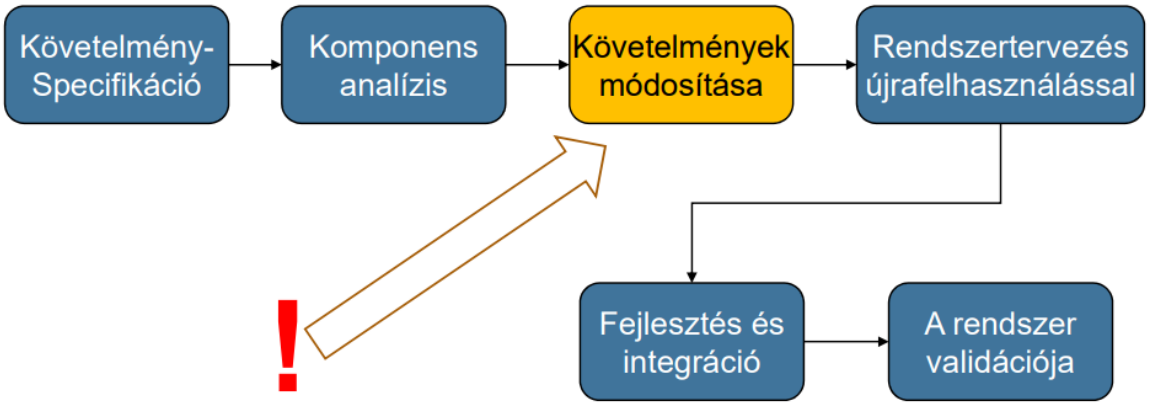
Ismertesse a vízesés modell működését. Milyen problémái vannak ennek a modellnek?
- Nehéz a változó megrendelői igényekhez igazodni, mert a projekt nehezen változtatható részegységekből áll. Ez a modell akkor hasznos, ha a követelmények

jól ismertek és csak nagyon kis változások lehetségesek a fejlesztés során. Sajnos csak kevés üzleti rendszernek vannak stabil követelményei.

Ismertesse az inkrementális fejlesztési modellt.



Ismertesse az újrafelhasználás-alapú fejlesztési modellt.



Milyen kérdésekre keres választ a szoftverspecifikáció tevékenység?
- Milyen szolgáltatásokat várunk el a rendszertől, milyen kötöttségeket és kényszereket kell figyelembe venni a fejlesztés és üzemeltetés során.

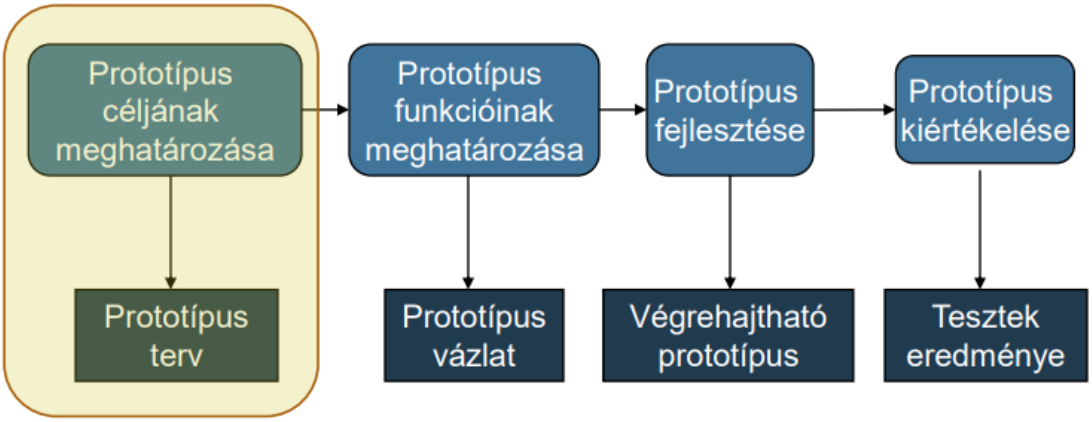
Mi célt szolgál a rendszerek validációja?
- A verifikáció és validáció célja annak bizonyítása, hogy a rendszer teljesíti a specifikációban foglaltakat és a felhasználó igényeinek megfelelően működik.

Sorolja fel a tesztelési eljárás 3 lépését. Ismertesse ezeket.
- Komponens- és részegység-tesztelés: A különálló komponenseket egymástól függetlenül teszteljük, a komponensek lehetnek: függvények, objektumok, vagy ezek összetartozó csoportjai. Fejlesztők: kódolás-tesztelés együtt
- Rendszertesztelés: a rendszer egészének tesztelése, komponensek együttműködése okozhat problémát, hibakeresés, nagy rendszereknél több lépéses lehet.

- Végteszt (átadási teszt, alfa-teszt): a megrendelő által szolgáltatott valós adatokkal annak ellenőrzése, hogy a megrendelő igényeit valóban kielégíti, béta teszt: alfa-teszten átment rendszer tesztelése potenciális felhasználókkal

Mit jelent a szoftver evolúciója?
- Ahogy a változó üzleti-gazdasági körülmények miatt a követelmények változnak, a kiszolgáló szoftvernek is változnia és fejlődnie kell. Bár a fejlesztés és karbantartás között régebben éles határvonal húzódott, ez egyre kevésbé releváns, hiszen egyre kevesebb a teljesen új rendszer (evolúció).

Ismertesse az eldobható prototípussal való fejlesztés menetét.

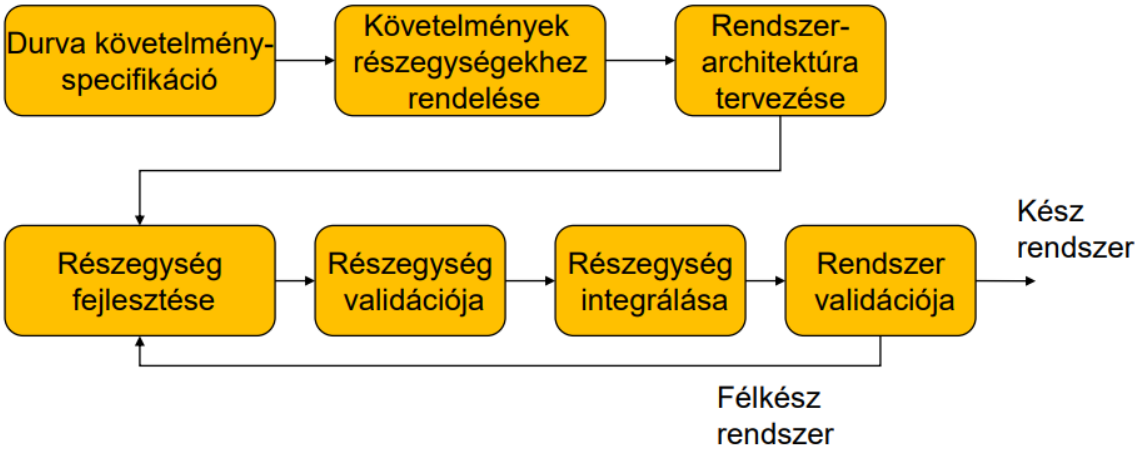


Soroljon fel 3 eldobható prototípus fajtát.
- Program, Papír prototípus, „Óz a nagy varázsló” prototípus

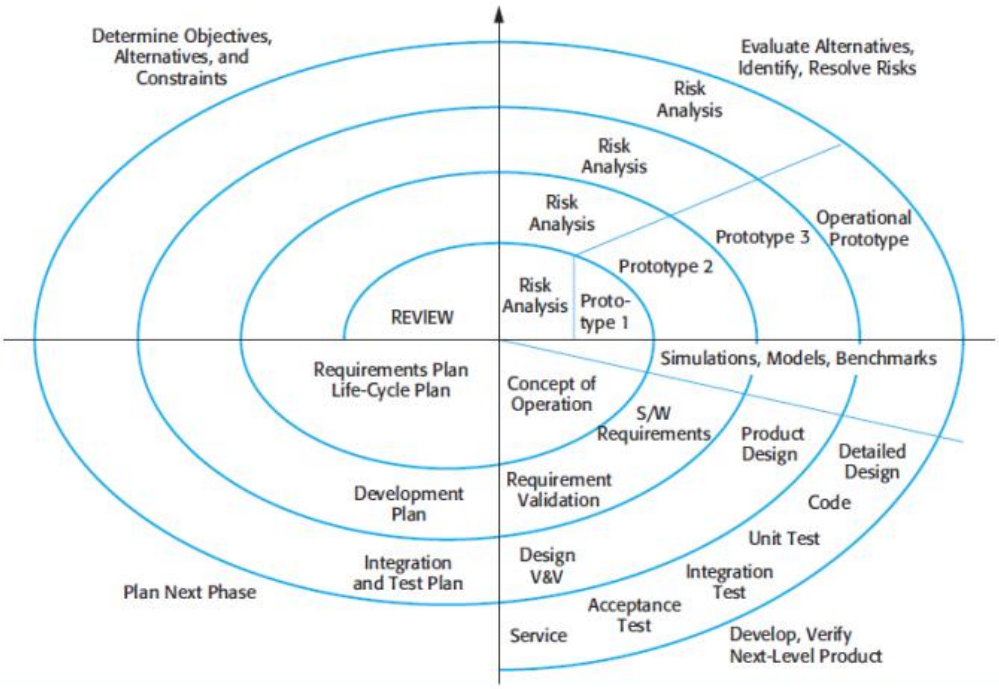
Mit jelent a papír prototípus?
- Rövid vázlat szerű általában rajz prototípus

Mit jelent az „Óz a nagy varázsló” prototípus?
- Kicsi, a valós programra hasonlító, azt imitáló egyszerű és gyors program

Ismertesse az inkrementális fejlesztés menetét, működését.



Ismertesse a spirális fejlesztési modell menetét, működését.



Mit jelent a RUP iterációja?
- Iteráció = mini-projekt. Rövid (általában 2-4 hét), előre meghatározott hosszúságú. A fejlesztés ilyen iterációk sorozata. A rendszer fokozatosan nő és finomodik.

Mit jelent RUP-ban visszacsatolás és az adaptivitás?
- Visszacsatolás: fejlesztési folyamat során rendszeresen értékelik az elkészült termékeket, eredményeket és az aktuális helyzetet.
- Adaptáció: a projekt folyamán a kapott visszacsatolás alapján módosítják a fejlesztési tervet, a követelményeket vagy akár az architektúrát is.

Sorolja fel az iteratív fejlesztés előnyeit.
- A nagy kockázatú elemek korai ellenőrzése (technikai problémák, követelmények, használhatóság).
- Korai, jól látható előrehaladás a fejlesztésben.
- Korai felhasználói visszacsatolás, ami az igényeknek jobban megfelelő rendszerhez vezet.
- Kezelhető komplexitás: a fejlesztőket nem terheli le az „analízis paralízis”, vagy egyéb hosszú és bonyolult fejlesztési lépések.
-Az iterációkon belül nyert tapasztalatok felhasználhatók a fejlesztési folyamat javítására

Mit jelent a RUP-ban a „time-boxing”?
- időkorlátos fejlesztés

Sorolja fel a RUP fázisait.
- Alapozás: a rendszer számára egy üzleti modell megalkotása.
- Kidolgozás: a problématér megértése és a rendszer-architektúra kidolgozása.
- Konstrukció: rendszertervezés, programozás és tesztelés.
- Átadás: a rendszer telepítése a működési környezetbe.

Mit jelent, ha egy fejlesztési metodika „könnyű/nehéz”?
- Nehéz: sok termék, bürokratikus atmoszféra, szigorú ellenőrzés, részletes tervezés.

- Könnyű: ennek ellentéte

Mit jelent, ha egy fejlesztési metodika „prediktív/adaptív”?
- Prediktív: hosszú időre részletesen előre próbál tervezni, általában vízesés-modell alapon.
- Adaptív: változások beépítése, azokhoz adaptálódás

Mit jelent, ha egy fejlesztési metodika „agilis”?
- könnyű és adaptív

Mit tartalmaz az Agilis kiáltvány?
- Az egyéneket és a személyes kommunikációt biztosítanak a módszertanokkal szemben. Működő szoftvert a dokumentációval szemben. A megrendelővel történő együttműködést a szerződéssel szemben. A változás iránti készséget a tervek helyett.

Sorolja fel az agilis fejlesztési módszertanok 5 közös alapelvét. Magyarázza el ezeket.
- Megrendelő bevonása, inkrementális teljesítés, emberek a folyamat előtt, változások befogadása, egyszerűségre törekvés

Hogyan támogatja az XP az inkrementális fejlesztést?
- Apró, gyakori bővítések, napi több verzió, kb. kéthetente release. A követelmények egyszerű történetekként (USER STORY) jelennek meg (hogyan használja a felhasználó a rendszert egy tipikus feladat során): STORY CARD. A felhasználó adja meg ezek prioritását, de üzleti szempontból hasznosabb feladatok nagyobb prioritásúak. A STORY CARD-ból a fejlesztők kisebb feladatokat (TASK) definiálnak.

Hogyan támogatja az XP a megrendelői jelenlétet?
- Állandó jelenlét a fejlesztői csapatban, User Story fejlesztése, prioritások definiálása, elfogadási teszteket a megrendelő képviselője definiálja

Mit jelent az XP-ben a páros programozás?
- Egy gép, 2 ember. Egymás támogatása: ötletek a megvalósításhoz, felügyelet (informális szoftvervizsgálat). A kódbázis közös. Mindenki felelős a teljes kódért. Nincsenek elhatárolt „szakértői szigetek”. Bárki módosíthatja a kód bármely részét. Fenntartható fejlesztési folyamat: túlóra mentes, vonzó munkakörülmények

Mit jelent a „test driven” fejlesztés?
- test-driven (tesztvezérelt) fejlesztés. Először a tesztet kell implementálni, utána jöhet a funkciót megvalósító kód, automatikus tesztelési környezet.

Ismertesse a Scrum módszertan Product Owner szereplőjének szerepét.
- A „megrendelő hangja”. Maximális üzleti értéket igyekszik elérni. A követelményeket ő állítja elő, prioritással (Product Backlog). Elfogadási kritériumokat ő definiálja. Sprint végén elfogadja a terméket.

Ismertesse a Scrum módszertan Scrum Master szereplőjének szerepét.
- Felügyeli a Scrum szabályok betartását. Szükség esetén tanítja a csapatot. Nyugodt munkakörülményeket biztosít a csapatnak („árnyékol”). Akadályok elhárításáért felel. Nem feltétlenül technikai hátterű szakembe

Ismertesse a Scrum módszertanban a Scrum Team szerepét.
- Megértik a PO által felállított követelményeket. Saját hatáskörben osztják fel a munkát. Szállítják a követelményeknek megfelelő inkrementumot

Ismertesse a Scrum módszertanban a sprintek működését.

- Timeboxing (2-4 hét). Product Backlogból feladatok elővétele. Scrum team felosztja a munkát. Háborítatlanul dolgoznak a sprint teljes ideje alatt. Sprint végén demó a Product Ownernek. Sprint végén visszatekintés a sprintben szerzett tapasztalatokra: a következő sprintekben szükség szerint módosítás

Mit tartalmaz a Product Backlog?
- Követelmények, hibajavítások, feladatok, ötletek, fejlesztési javaslatok. Priorítási sorrendben

Mit tartalmaz a Scrum Tábla?
- Megvalósítandó feladatok (task-okra bontva) és azok állapotát (To Do, in progress, testing, done)

Ismertesse a napi megbeszélések működését a Scrum módszertanban.
- Fókuszált, (állva,) rövid ideig tart (time boxing!). Itt osztja meg a csapat egymással az előrehaladást és az esetlegesen tapasztalt akadályokat

Mit írnak le a követelmények?
- Milyen szolgáltatásokat nyújtson, milyen peremfeltételek mellett

Mi a követelményelemzés?
- A felhasználói igények és a rendszer működési feltételek feltárásának folyamata

Mi a követelmények két alapvető szintje?
- Felhasználói követelmények, rendszerkövetelmények

Ismertesse a felhasználói követelmények fő ismérveit.
- A rendszer szolgáltatásairól és a működési feltételekről szóló természetes nyelven írt állítások és diagramok. Az ügyfél számára készül.

Ismertesse a rendszerkövetelmények fő ismérveit.
- Strukturált dokumentum, amely tartalmazza a rendszer funkcióinak, szolgáltatásainak és működési feltételeinek részletes leírását. Definiálja az implementálandó feladatokat, így a megrendelő és a szállító közti szerződés része lehet.

Mit jelent a funkcionális és nem funkcionális követelmény?
- Funkcionális követelmények: milyen szolgáltatásokat kell a rendszernek nyújtania, hogyan kell bizonyos bemeneti adatokra reagálnia és hogyan kell viselkednie egyes helyzetekben.
- Nem funkcionális követelmények: a rendszer szolgáltatásaira és funkcióira vonatkozó feltételek és kényszerek. Pl. Időzítési kényszerek, a fejlesztésre vonatkozó kényszerek, szabványok, stb.

Adjon példát funkcionális követelményekre a NEPTUN rendszerben.
- Pl.: kezelje a vizsgákat, vizsgajelentkezéseket, vizsga értékeléseket, újra vizsgázás stb.

Adjon példát nem funkcionális követelményekre a NEPTUN rendszerben.
- Feleljen meg az oktatási szabványoknak

Mit jelent, hogy egy követelményrendszer teljes?
- Minden szükséges igény leírását tartalmazza.

Mit jelent, követelményrendszer konzisztens?
- Nem lehet konfliktus vagy ellentmondás az igények megfogalmazásában.

Sorolja fel a nem funkcionális követelménye 3 alapvető fajtáját. Mindegyikre adjon 1-1 példát.
- Termék követelmények: olyan követelmények, amelyek meghatározzák a termék viselkedési módját. Pl. végrehajtási sebesség, megbízhatóság, stb (valós példa: A leállási idő munkaidőben nem haladhatja meg a napi 5 másodpercet.).
- Szervezeti követelmények: a szervezet stratégiájából és működési módjából következő követelmények. Pl. felhasznált szabványok, implementációs követelmények, stb. (valós példa: a rendszerfejlesztés és a dokumentáció a XYZCo-SP-STAN-95 szabvány szerint kell történjen.)
- Külső követelmények: a rendszer és a fejlesztési eljárás szempontjából külső tényezők hatására fellépő követelmények. Pl. más rendszerekkel való együttműködés, jogi szabályozás, stb. (valós példa: Az operátorok nem juthatnak hozzá a rendszer felhasználóinak személyes adataihoz a nevükön és azonosítóikon kívül.)

Mi a követelmény-dokumentum?
- A követelmény-dokumentum egy hivatalos dokumentum, amely tartalmazza, hogy mit várunk a rendszer fejlesztőitől. Mind a felhasználói követelményeket, mind a rendszerkövetelmények specifikációját tartalmaznia kell.

Mi a megvalósíthatósági tanulmány?
- A megvalósíthatósági tanulmány dönti el, hogy érdemes-e a rendszert fejleszteni. Rövid, célirányos tanulmány arról, hogy:
    1. hozzájárul-e a rendszer a szervezet célkitűzéseinek eléréséhez
    2. a rendszer a jelenlegi technológiával és pénzügyi kerettel megvalósítható-e
    3. a rendszer integrálható-e a jelenleg használatos többi rendszerrel.

Sorolja fel a követelmények megfogalmazásának 4 alapvető módszerét.
- Természetes nyelv (minden mondat egy követelmény).
- Strukturált természetes nyelv (űrlap (template) alkalmazása: minden mező egy előre definiált aspektus).
- Grafikus leírás (grafikus modell szöveges kiegészítéssel, pl. UML use case-ek és szekvencia diagramok).
- Matematikai leírások (matematikai precizitású modellek, pl. állapotgépek)

Mik a természetes nyelvi leírások szabályai? Mik ezen leírás hátrányai?
- Legyen egy állandó formátumunk a követelmények számára. Pl. minden követelmény egy mondat, melyet zárójelben a magyarázat követ (miért szükséges, fontos). Használjuk a kifejezéseket következetesen. Pl. „kell” a szükséges, a „javallott” a kívánatos követelmények jelzésére. Használjunk szövegkiemeléseket a követelmény fontos részeinek jelzésére. Ne használjunk számítógépes zsargont.
- Hátrányai: többértelműség (a követelmények írói és olvasói ugyanazt kell értsék a szavakon. Atermészetes nyelvek eredendően két (vagy több) értelműek, így ez nehézkes.) Túlságos flexibilitás (ugyanazt a dolgot sokféleképpen lehet elmondani). A modularizálhatóság hiánya (a természetes nyelvi struktúrák nem alkalmasak a rendszerkövetelmények strukturálására).

Mit jelent a strukturált nyelvi specifikáció? Milyen fő elemeit szoktuk alkalmazni?
- A követelmény-író szabadságát előre definiált sablonok korlátozzák. Minden követelményt egy standard módon írunk meg. A leírásban használható terminológia korlátozva lehet.

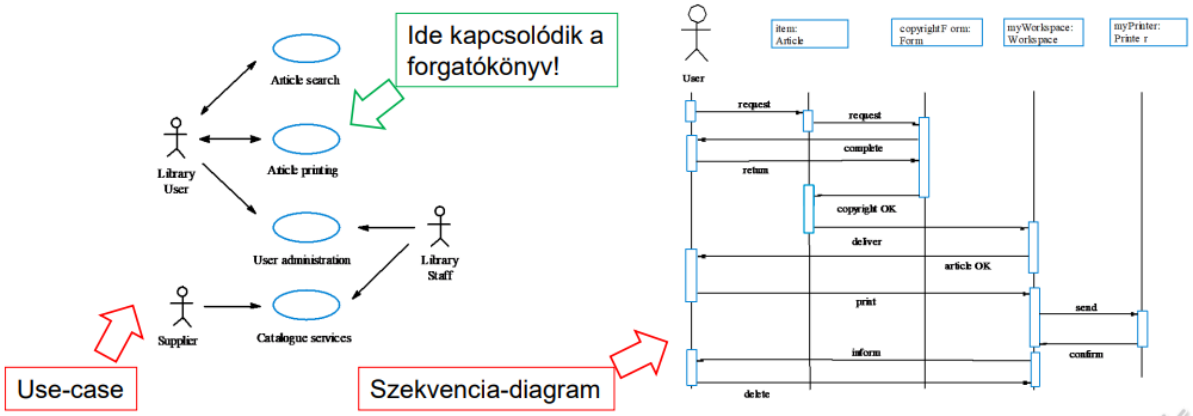
- Fő elemei: A funkció vagy entitás definíciója. A bementek leírása + honnak erednek. A kimenetek leírása + hová mennek. Más felhasznált entitások felsorolása. Elő- és utófeltételek (pre-, post-condition). Mellékhatások leírása.

Sorolja fel a követelményfeltárási eljárás 4 lépését.
- 1. Követelmények feltárása: az érdekeltekkel való interakció során fel kell fedni igényeiket. Az alkalmazási környezet követelményeit is ebben a fázisban kell feltárni.
- 2. Követelmények osztályozása és szervezése: a kapcsolódó követelmények csoportosítása és koherens rendszerbe szervezése.
- 3. Prioritások, tárgyalások: a követelmények fontossági sorrendbe állítása. A konfliktusok feloldása.
- 4. Követelmények dokumentálása: a követelmények dokumentálása. Ez lesz a spirál következő körének bemenete.

Sorolja fel a követelményfeltárás 4 lehetséges módszerét.
- Interjúk (érdekeltekkel): zárt: egy előre meghatározott kérdés-csoportra kell válaszolni. Nyílt: nincs előre meghatározott menetrend, a megválaszolandó problémákat az érdekeltekkel együtt tárjuk fel.
- Forgatókönyvek (szenáriók): valós példák arról, hogy hogyan kell a rendszert használni. Nem arról szól, hogy mi az elvárás, hanem arról, hogy a rendszert hogyan tervezzük használni
- Használati esetek (use case-ek) : UML, Grafikus modellek + forgatókönyvek
- Etnográfia: dolgozók munka közbeni megfigyelése („Nagy testvér”). Nem kell elmagyarázni a munkafolyamatot. Fontos társadalmi és szervezeti tényezőkre derülhet fény. Prototípussal együtt: célzott etnográfia

Mi a forgatókönyv, milyen információkat kell tartalmaznia?
- A forgatókönyvek (szenáriók) valós életből vett példák arról, hogy hogyan kell a rendszert használni. Tartalmazniuk kell: a kiinduló szituáció leírását, az események normál menetének leírását, annak leírását, hogy mi sikerülhet rosszul: kivételkezelés, információt már párhuzamos tevékenységekről, a szcenárió befejezése utáni állapot leírását.

Mi a use case? Rajzoljon fel egy tetszőleges use-case-t UML jelölések felhasználásával.
- Esettanulmányok (use cases). Szcenárió-alapú technika, az UML része. Azonosítja az interakcióban részt vevő aktorokat, funkciókat, és leírja magát az interakciót is (forgatókönyvvel).



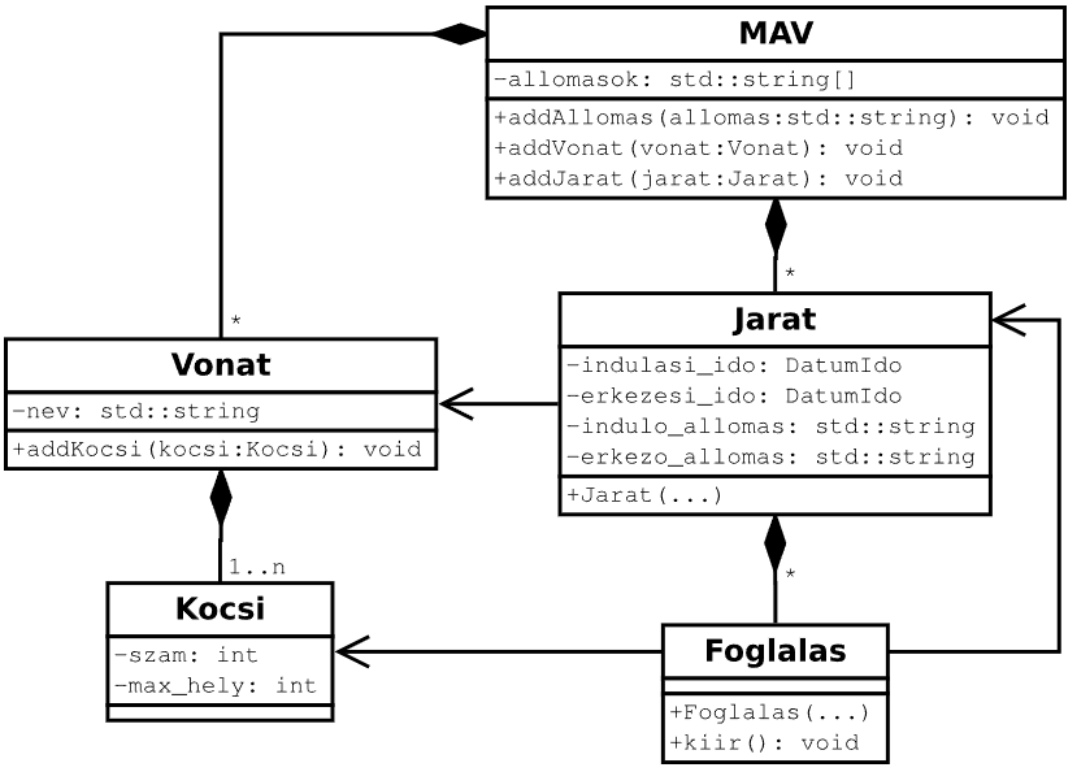
Soroljon fel 3 technikát követelmények ellenőrzésére.
- Követelmény szemle: a követelmények szisztematikus kézi ellenőrzése.

- Prototípus készítése: a rendszer végrehajtható modelljének segítségével ellenőrizzük a követelményeket.
- Tesztek készítése (tervezése): tesztelhetőség ellenőrzése követelmény-tesztek kidolgozásával.

A követelmények mely 4 kívánt tulajdonságát ellenőrizzük a követelmény-validáció során?
BTW ez 5
- Érvényesség: a rendszer a megrendelő igényeit legjobban kielégítő szolgáltatásokat nyújtja?
- Konzisztencia: vannak a követelmények között ellentmondások, konfliktusok?
- Teljesség: a megrendelő számára minden szükséges funkció rendelkezésre áll?
- Realitás: a jelenlegi technológiával és költségvetéssel implementálható a rendszer?
- Verifikálhatóság: ellenőrizhetők a követelmények?

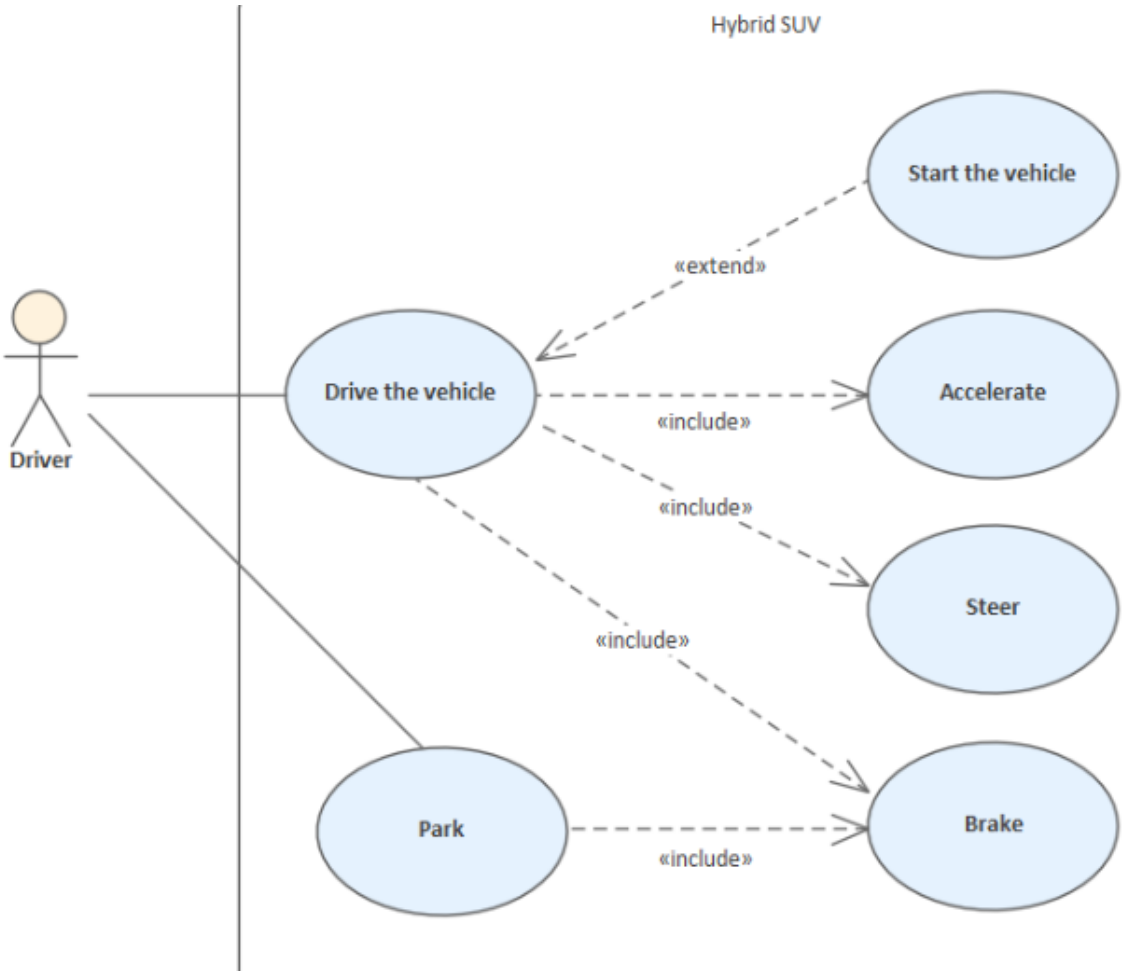
Mire szolgálnak a kontextus modellek?
- A kontextus modellek a rendszer működési környezetét mutatják be: mi van a rendszer határain kívül. Társadalmi és szervezeti érdekek befolyásolhatják a döntést, hogy hová helyezzük a rendszer határait. Achitektúrális modellekkel a rendszert és annak más rendszerekkel való kapcsolatát mutatjuk be.

Rajzoljon fel egy tetszőleges kontextus modellt UML jelöléssel.



Mit tartalmaz egy UML use case diagram?
- Use-case (használati eset): rendszer és környezete közötti interakciók. Rendszer – felhasználók, más rendszerek

Rajzoljon fel egy tetszőleges use case diagamot.



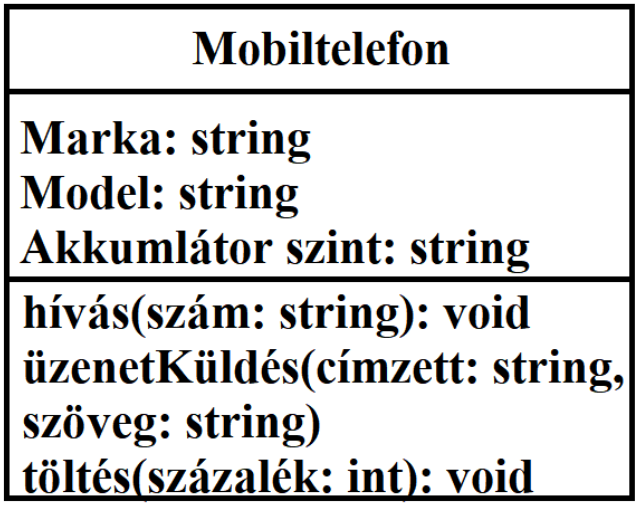
Magyarázza el az UML szekvencia-diagramm elemeinek jelentését. Mi található a felső sorban? Mit jelképez a pálcikaember? Mik a dobozok? Mit jelentenek a függőleges szaggatott vonalak? Mit jelent a vékony téglalap? Mit jelentenek a nyilak?

- Szereplők (felső sorban): Aktor (pl. recepciós)
- Pálcika emberke: felhasználó.
- Dobozok: objektumok.
- Szaggatott vonal: Idő fentről lefelé múlik.
- Téglalap: aktivitás.
- Interakció: nyilak, paraméterrel vagy visszatérési értékkel

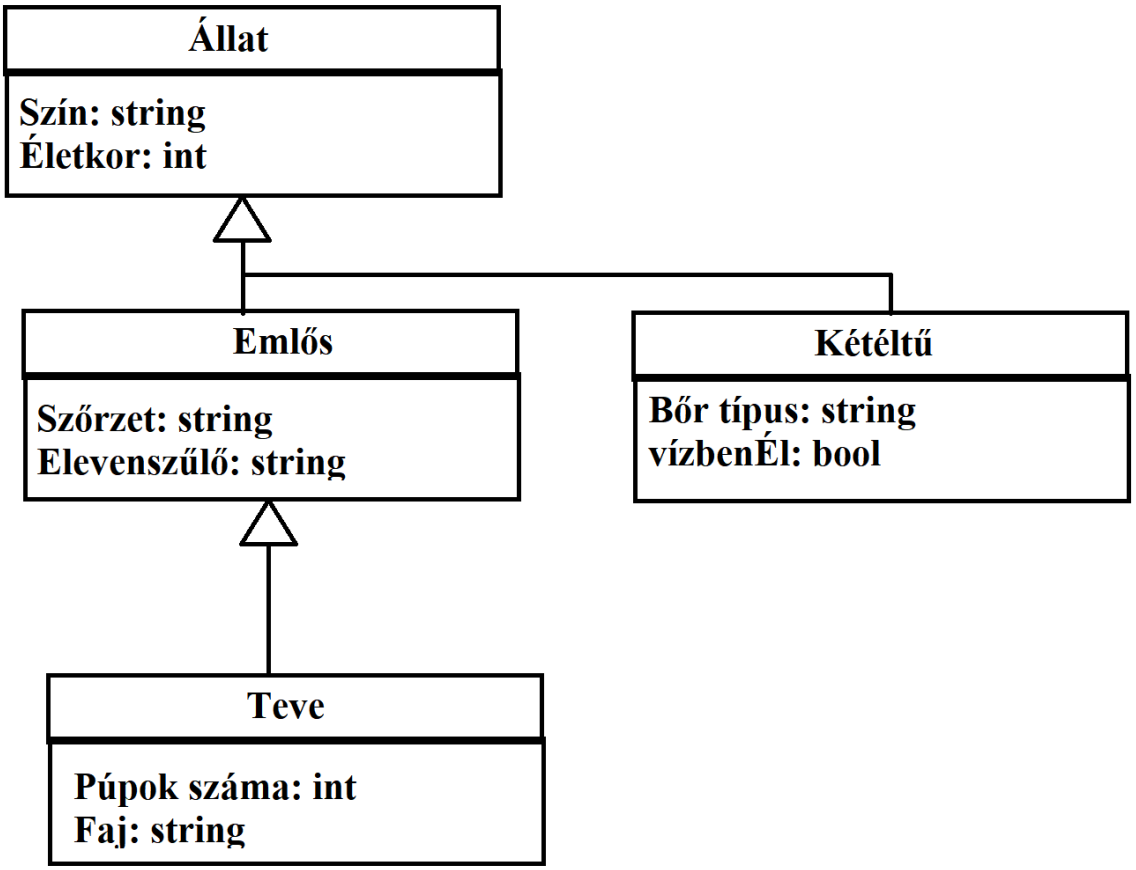
Magyarázza el az UML objektum jelölését.

- Az objektum osztályokat téglalapok jelképezik, amelyben a név felül, a tulajdonságok (attribútumok) középen, az operációk (metódusok) pedig az alsó részben helyezkednek el. Az objektumok közti relációkat (melyeket asszociációnak is neveznek) az objektumokat összekötő vonalak jelképezik.

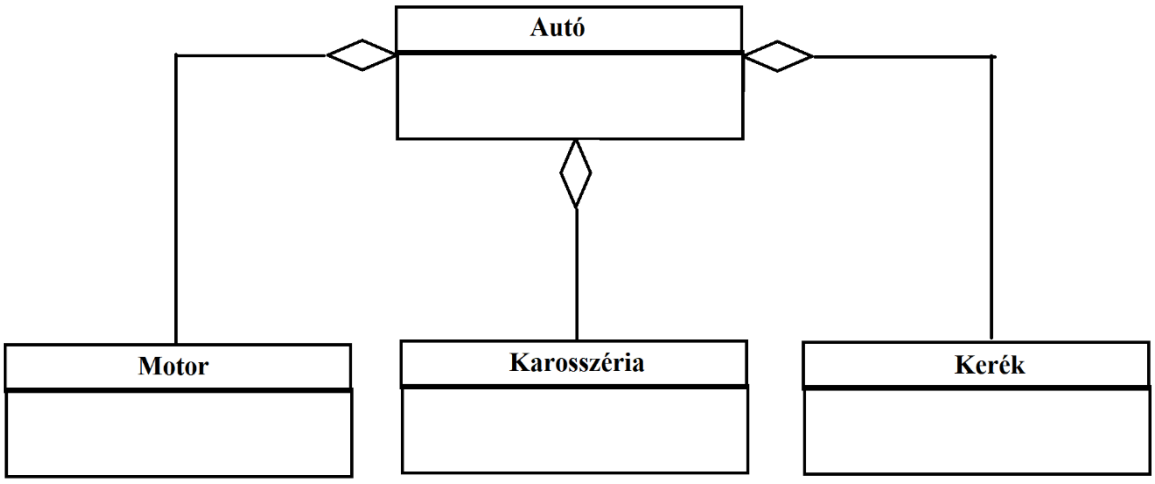
Rajzolja fel UML jelölésekkel egy MOBILTELEFON objektum modelljét. Legyen legalább 3 attribútuma és 3 metódusa.



Rajzoljon fel öröklési modellt, amely a következő osztályokat tartalmazza: ÁLLAT, EMLŐS, KÉTÉLTÚ, TEVE. Egészítse ki a modellt úgy, hogy minden osztályban szerepeljen 2-2 attribútum.



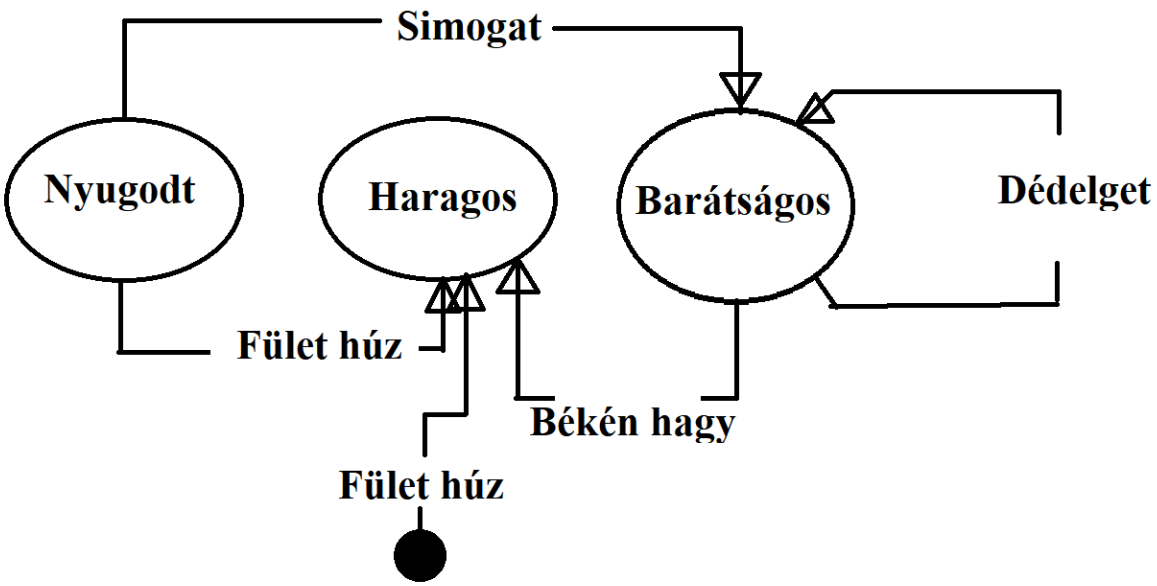
Rajzolja fel aggregációs modell segítségével a SZEMÉLYAUTÓ osztályt. A modell tartalmazza a MOTOR, KERÉK, FUTÓMŰ, KAROSSZÉRIA elemeket.



Mi a különbség az aggregáció és a kompozíció között?
- aggregáció: külön is léteznek (üres rombusz). kompozíció: külön nem léteznek (teli rombusz)

Magyarázza el az UML állapotgép jelöléseit. Mik a lekerített téglalapok, mik a nyilak, mik a nyilakra írt szövegek?
- A rendszer állapotai a csomópontok, köztük futó irányított élek pedig az események. Egy esemény bekövetkezésekor a rendszer egyik állapotból a másikba megy át. A modellnek kisebb részekre, al-modellekre bontását teszi lehetővé (dekompozíció). Az akciók rövid leírása az egyes állapotokban a 'do' utasítás után található. Kiegészíthető az állapotok és a gerjesztések leírását tartalmazó táblázatokkal.
- Lekerekített téglalapok: állapot
- Nyilak: állapotátmenetek
- Nyilakra írt szöveg: állapotátmenet eseménye

Készítse el a KUTYA működését leíró egyszerű állapotgépet. A KUTYA állapotai legyenek: NYUGODT, HARAGOS, BARÁTSÁGOS, míg az események a következők: SÍMOGAT, DÉDELGET, FÜLET_HÚZ, BÉKÉN_HAGY (az eseményeket a gazdi váltja ki).


Simogat
Nyugodt
Haragos
Barátságos
Dédelget
Fület húz
Békén hagy
Fület húz